

Predicting Spotify Track Popularity from Audio Features

A Supervised Learning Approach Using the Spotify Tracks Dataset

Submission 2: Proposal, Code, and Notebooks

Machine Learning Assignment — PS26 - DSC01 Machine Learning 120B

Balla Shivaram — Matriculation Number: 89235214

Project Title	Predicting Spotify Track Popularity from Audio Features
Dataset Name	Spotify Tracks Dataset (tracks.csv)
Number of Rows	899,702 (raw); ~899,200 (modeling subset)
Number of Columns	16 (raw); 22 (engineered features)
Target Variable	popular (binary): 1 if popularity ≥ 50 , else 0
Task Type	Binary Classification (Supervised Learning)
Class Balance	~89% non-popular, ~11% popular
Number of Research Questions	7

1. Project Title

Predicting Spotify Track Popularity from Audio Features: A Supervised Learning Approach Using the Spotify Tracks Dataset

2. Dataset

Name: Spotify Tracks Dataset (tracks.csv)

Original size: 899,702 rows x 16 columns (~130 MB CSV)

Modeling subset: ~899,200 rows x 22 engineered features (after removing rows with missing audio features)

Key columns used

- popularity — numeric Spotify popularity score (0-100); used to define the binary target
- tempo — estimated track tempo in BPM
- energy — perceptual measure of intensity and activity (0-1)

- danceability — how suitable a track is for dancing (0-1)
- valence — musical positiveness conveyed by a track (0-1)
- acousticness — confidence the track is acoustic (0-1)
- liveness — detects presence of audience in the recording (0-1)
- instrumentalness — predicts whether a track contains no vocals (0-1)
- speechiness — detects presence of spoken words (0-1)
- key — the estimated overall key of the track (0-11)
- mode — modality (major=1, minor=0)
- time_signature — estimated time signature
- genres — list of genre tags; used to derive 5 genre family dummies (Pop, Rock, Hip-Hop, Electronic, Other)

3. Target Variable

popular (binary, 0 or 1)

Definition: popular = 1 if popularity $\geq$ 50, else 0

Justification: Spotify's popularity score (0-100) is computed from total plays and recency of plays. A threshold of 50 splits the catalogue into a meaningful 'popular' minority (the top ~11%) versus the long tail of niche, old, or undiscovered tracks. This threshold is widely used in academic studies of music popularity prediction and creates a non-trivial binary classification task with substantial class imbalance, making it suitable for testing class-imbalance handling strategies (RQ6).

Class balance on the modeling subset: ~89% non-popular, ~11% popular. The strong skew motivates explicit imbalance-handling experiments.

4. Type of Task

Binary Classification (Supervised Learning).

5. Problem Statement

The global music streaming industry generates over \$30 billion in annual revenue, with Spotify alone serving over 600 million users and hosting more than 100 million tracks. A small fraction of these tracks attracts the majority of listening time, while the long tail remains largely undiscovered. Understanding which audio characteristics predict popularity is valuable for artists, labels, playlist curators, and the platform itself.

This project addresses the question: given purely the audio features of a track (tempo, energy, danceability, valence, acousticness, instrumentalness, etc.) and its genre tags, can we predict whether it will achieve a popularity score of at least 50 on Spotify?

Unlike content-based recommendation that requires user-track interaction histories, this project uses only intrinsic track properties available before release. The findings aim to identify which audio features most strongly drive popularity, where prediction succeeds and fails, and how to handle the substantial class imbalance inherent in popularity data.

6. Research Questions

This project is guided by the following seven research questions, each addressing a distinct aspect of the supervised learning problem:

- RQ1: How consistent are model performance estimates between a single stratified hold-out split and 5-fold stratified cross-validation?
- RQ2: Do gain-based and permutation-based feature importance methods agree on which audio features most drive popularity prediction?
- RQ3: Does popularity rate and model predictability vary systematically across tempo tiers (Slow / Medium / Fast / Very Fast)?
- RQ4: Does a model trained on the full dataset predict popularity equally well across different genre families (Pop, Rock, Hip-Hop, Electronic, Other)?
- RQ5: How do energy and danceability jointly affect popularity rate and model predictability — and is there a significant interaction effect?
- RQ6: How do class imbalance mitigation strategies (SMOTE, undersampling, class weighting) affect the precision-recall trade-off given the strong skew toward non-popular tracks?
- RQ7: Does acousticness tier (Electronic / Mixed / Acoustic) influence popularity predictability?

7. Proposed Methodology

Stage 1: Data acquisition and inspection

- Load the Spotify Tracks CSV (~899K rows).
- Inspect schema, dtypes, and missingness patterns across all 16 columns.
- Document data quality observations and identify columns suitable for feature engineering.

Stage 2: Filtering and target definition

- Remove rows with missing audio features (~500 rows), leaving ~899,200 records.
- Define binary target: popular = 1 if popularity ≥ 50 , else 0.
- Verify class balance (~89% non-popular vs ~11% popular — motivates RQ6 imbalance experiments).

Stage 3: Feature engineering

- Raw audio features (12): tempo, energy, danceability, valence, acousticness, liveness, instrumentalness, speechiness, key, mode, time_signature.
- Engineered combinations (5): loudness_proxy = energy * (1 - acousticness), valence_x_energy = valence * energy, is_high_energy, is_danceable, is_acoustic, is_instrumental.
- Genre family dummies (5): genre_pop, genre_rock, genre_hiphop, genre_electronic, genre_other (from substring matching on the genres field).
- Final feature matrix: 22 numeric features ready for sklearn-compatible models.

Stage 4: Train-test split

- For tractable runtime, work with a 100,000-row sample drawn with random_state=42.
- Stratified 80/20 split using sklearn (random_state=42) preserving the ~11% popular rate.

Stage 5: Model training

- Five supervised classifiers: Logistic Regression, SVM (RBF), Random Forest, Gradient Boosting, XGBoost.
- Linear/kernel models receive standardised features (StandardScaler); tree-based models use raw features.

Stage 6: Evaluation

- Predictions evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- Per-segment evaluation for RQ3 (tempo tiers), RQ4 (genre families), RQ5 (energy x danceability cells), RQ7 (acousticness tiers).
- Four imbalance strategies compared on the same held-out test set for RQ6.

Stage 7: Visualisation and reporting

- Each RQ produces one publication-grade figure (PDF + PNG) and one results table (CSV).
- All seven notebooks are self-contained and runnable on Kaggle without modification.

8. Machine Learning Models

Model	Library	Family	Role
Logistic Regression	sklearn.linear_model	Linear	Interpretable linear baseline

SVM (RBF kernel)	sklearn.svm	Kernel	Non-linear baseline
Random Forest	sklearn.ensemble	Bagging	Variance-reducing tree ensemble
Gradient Boosting	sklearn.ensemble	Boosting	Sequential tree ensemble
XGBoost	xgboost	Boosting	State-of-the-art gradient boosting

9. Evaluation Metrics

Metric	Definition	Why it matters
Accuracy	$(TP + TN) / \text{total}$	Overall correctness; misleading on imbalanced data
Precision	$TP / (TP + FP)$	Of tracks predicted popular, how many truly are
Recall	$TP / (TP + FN)$	Of actual popular tracks, how many the model catches
F1-Score	Harmonic mean of precision and recall	Balances precision and recall; primary headline metric
ROC-AUC	Area under ROC curve	Threshold-independent ranking quality measure

10. Expected Figures and Tables

RQ	Figure	Table
RQ1	Grouped bar chart: hold-out vs 5-fold CV per model with error bars	table_rq1_cv_stability.csv
RQ2	Side-by-side horizontal bars: gain-based vs permutation importance	table_rq2_permutation_importance.csv
RQ3	Two panels: popularity rate by tempo tier + model performance bars	table_rq3_tempo_tier_popularity.csv
RQ4	Two panels: popularity rate & volume by genre family + Accuracy/F1/AUC bars	table_rq4_genre_family.csv

RQ5	3x3 heatmaps: popularity rate and F1 by Energy x Danceability cells	table_rq5_energy_danceability_interaction.csv
RQ6	Grouped bar chart: four imbalance strategies x five metrics	table_rq6_imbalance_strategies.csv
RQ7	Two panels: popularity rate by acousticness tier + F1/AUC bars	table_rq7_acousticness_tiers.csv

11. Tools and Infrastructure

- Programming language: Python 3.10+
- Numerical computing: NumPy, Pandas
- Machine learning: scikit-learn, XGBoost, imbalanced-learn
- Visualisation: Matplotlib
- Notebook environment: Jupyter (executable on Kaggle without modification)

12. Deliverables

- This proposal document (PDF + DOCX)
- Seven Jupyter notebooks (RQ1.ipynb through RQ7.ipynb), each self-contained and reproducible
- Seven figure PDFs (with PNG previews) and seven CSV result tables
- README.md, Methodology.md, and requirements.txt